

Lecture 10. The Perceptron

COMP90051 Statistical Machine Learning

Lecturer: Jean Honorio



THE UNIVERSITY OF
MELBOURNE

This lecture

- Perceptron model
 - * Introduction to Artificial Neural Networks
 - * The perceptron model
- Perceptron training rule
 - * Stochastic gradient descent
- Kernel perceptron

The Perceptron Model

A building block for artificial neural networks, yet another linear classifier

Biological inspiration

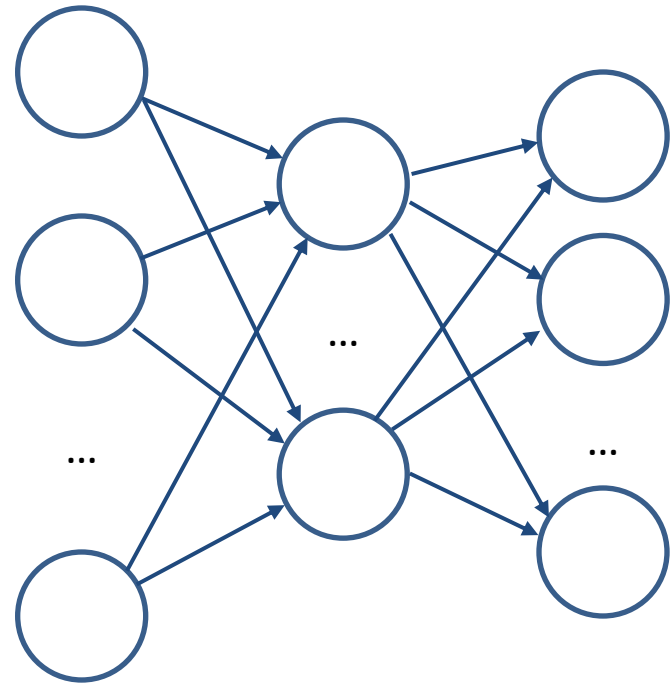
- Humans perform well at many tasks that matter
- Originally neural networks were an attempt to mimic the human brain

photo: Alvesgaspar,
Wikimedia Commons, CC3



Artificial neural network

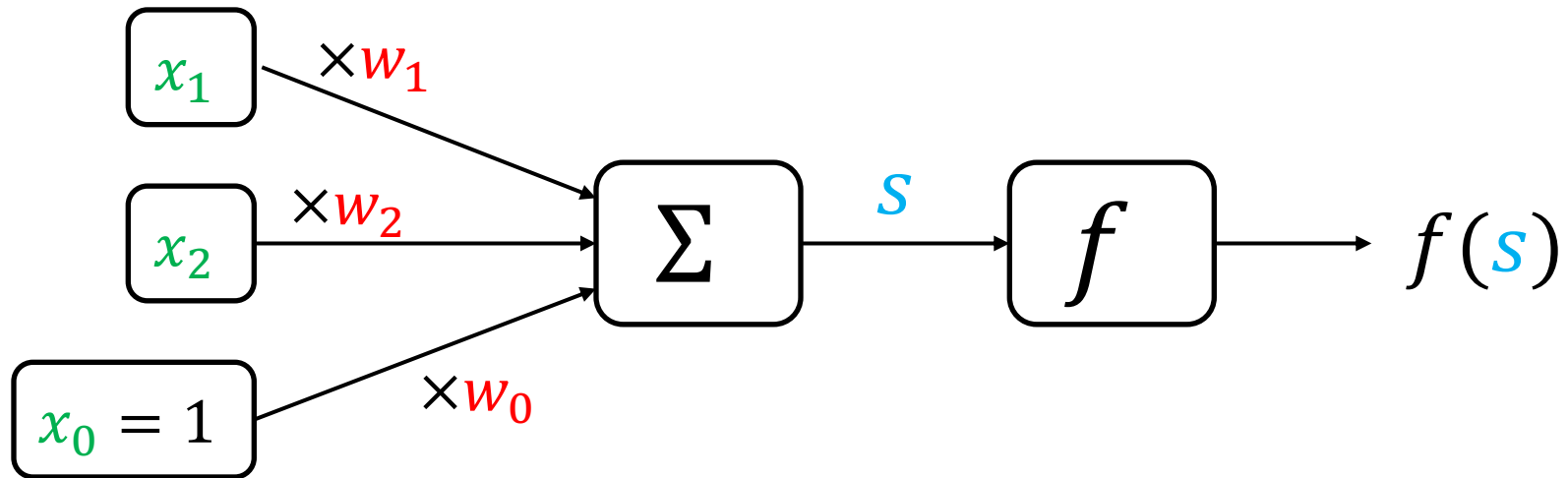
- As a *crude approximation*, the human brain can be thought as a mesh of interconnected processing nodes (neurons) that relay electrical signals
- **Artificial neural network** is a network of processing elements
- Each element converts inputs to output
- The output is a function (called **activation function**) of a weighted sum of inputs



Outline

- In order to use an ANN we need (a) to design network topology and (b) adjust weights to given data
 - * In this subject, we will exclusively focus on task (b) for a particular class of networks called **feed forward** networks
- Training an ANN means adjusting **weights** for training data given a pre-defined network **topology**
- First we will turn our attention to an individual network element, before building deeper architectures

Perceptron model



Compare this
model to logistic
regression

- x_1, x_2 – features/inputs
- w_1, w_2 – synaptic weights
- w_0 – bias weight
- f – activation function

Perceptron is a linear binary classifier

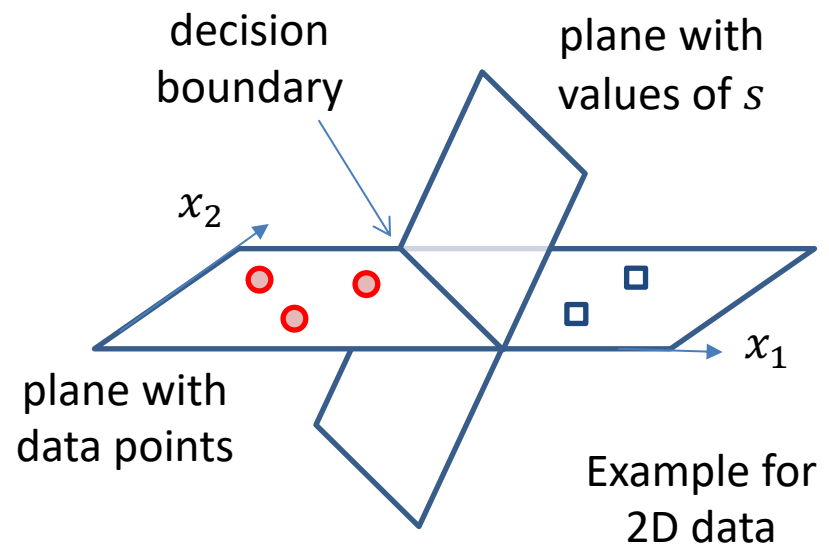
Perceptron is a
binary classifier:

Predict class A if $s \geq 0$

Predict class B if $s < 0$

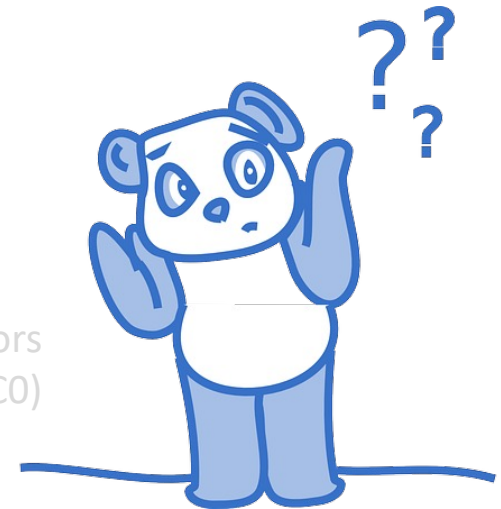
where $s = \mathbf{w}'\mathbf{x} = \sum_{j=0}^m w_j x_j$

Perceptron is a linear classifier: s
is a linear function of inputs, and
the decision boundary is linear



Exercise: find weights of a perceptron capable of perfect classification of the following dataset

x_1	x_2	y
0	0	Class B
0	1	Class B
1	0	Class B
1	1	Class A



art: OpenClipartVectors
at pixabay.com (CC0)

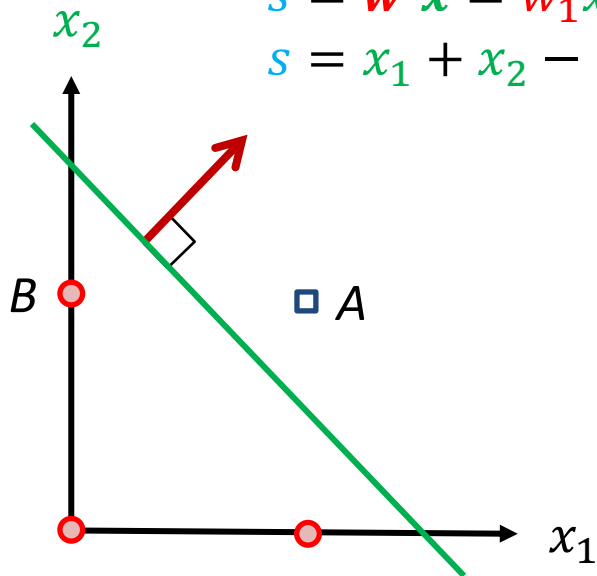
Exercise: find weights of a perceptron capable of perfect classification of the following dataset

x_1	x_2	y	s
0	0	Class B	-1.5
0	1	Class B	-0.5
1	0	Class B	-0.5
1	1	Class A	0.5

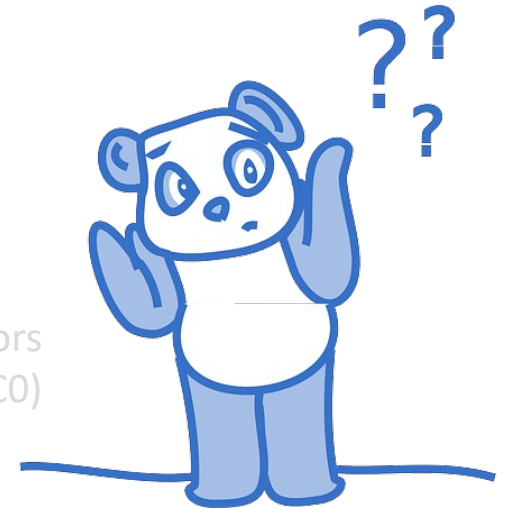
$$w_1 = 1, w_2 = 1, w_0 = -1.5$$

$$s = \mathbf{w}'\mathbf{x} = w_1x_1 + w_2x_2 + w_0$$

$$s = x_1 + x_2 - 1.5$$



art: OpenClipartVectors
at pixabay.com (CC0)



Mini Summary

- Perceptron
 - * Introduction to Artificial Neural Networks
 - * The perceptron model

Next: Perceptron training

Perceptron Training Rule

Gateway to stochastic gradient descent.

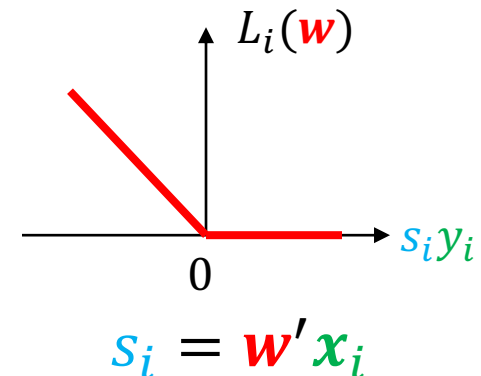
Convergence guaranteed by convexity.

Loss function for perceptron

- “Training”: finds weights to minimise some loss. Which?
- Our task is binary classification. Encode one class as $+1$ and the other as -1 . So each training example is now $(\mathbf{x}_i, y_i)$, where y_i is either $+1$ or -1
- Recall that, in a perceptron, $s_i = \mathbf{w}' \mathbf{x}_i = \sum_{j=0}^m w_j x_{ij}$, and the sign of s_i determines the predicted class: $+1$ if $s_i > 0$, and -1 if $s_i < 0$
- Consider a single training example.
 - * If y_i and s_i have same sign then the example is classified correctly.
 - * If y_i and s_i have different signs, the example is misclassified

Loss function for perceptron

- The perceptron uses a loss function in which there is no penalty for correctly classified examples, while the penalty (loss) is equal to s_i for misclassified examples*
- Formally:
 - * $L_i(\mathbf{w}) = 0$ if both s_i, y_i have the same sign ($s_i y_i > 0$)
 - * $L_i(\mathbf{w}) = |s_i|$ if both s_i, y_i have different signs ($s_i y_i < 0$)
- This can be re-written as $L_i(\mathbf{w}) = \max(0, -s_i y_i)$



* This is similar, but not identical to the SVM's *hinge* loss

Stochastic gradient descent

- Randomly shuffle/split all training examples in B **batches**
- Choose initial $\theta^{(1)}$
- For t from 1 to T 

Iterations over the entire dataset are called epochs
- For b from 1 to B
- Do gradient descent update using data from batch b
- Advantage of such an approach: computational feasibility for large datasets

Perceptron training algorithm

Choose initial guess $\mathbf{w}^{(0)}$, $k = 0$

For t from 1 to T (epochs)

For i from 1 to N (training examples)

Consider example $(\mathbf{x}_i, y_i)$

Update*: $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \nabla L_i(\mathbf{w}^{(k)})$

$k = k + 1$

$$L_i(\mathbf{w}) = \max(0, -s_i y_i)$$

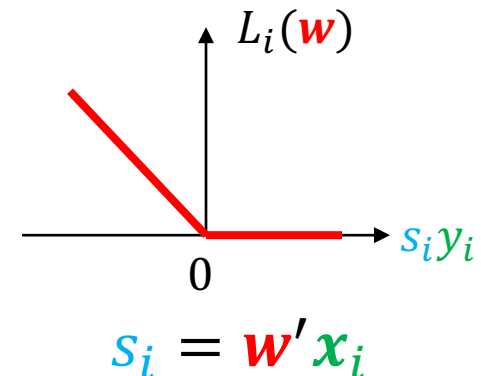
$$s_i = \mathbf{w}' \mathbf{x}_i$$

η is learning rate

*There is no derivative when $s_i = 0$, but this case is handled explicitly in the algorithm, see next slides

Perceptron training rule

- We have $\nabla L_i(\mathbf{w}) = \mathbf{0}$ when $s_i y_i > 0$
 - * We don't need to do update when sample i is correctly classified
- What is $\nabla L_i(\mathbf{w})$ when $s_i y_i < 0$?
 - * We need to update when sample i is misclassified
 - * We have $\nabla L_i(\mathbf{w}) = \mathbf{x}_i$ when $y_i = -1$ and $s_i > 0$
 - * We have $\nabla L_i(\mathbf{w}) = -\mathbf{x}_i$ when $y_i = 1$ and $s_i < 0$
 - * Thus $\nabla L_i(\mathbf{w}) = -y_i \mathbf{x}_i$



- $L_i(\mathbf{w}) = \max(0, -s_i y_i)$

Perceptron training algorithm

Choose initial guess $\mathbf{w}^{(0)}$, $k = 0$

For t from 1 to T (epochs)

For i from 1 to N (training examples)

Compute $s_i = (\mathbf{w}^{(k)})' \mathbf{x}_i$

If $s_i y_i \leq 0$: (sample i misclassified)

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta y_i \mathbf{x}_i$$

$$k = k + 1$$

($\eta > 0$ is called *learning rate*)

Perceptron training algorithm

Choose initial guess $\mathbf{w}^{(0)}$, $k = 0$

For t from 1 to T (epochs)

For i from 1 to N (training examples)

Compute $s_i = (\mathbf{w}^{(k)})' \mathbf{x}_i$

If $s_i y_i \leq 0$: (sample i misclassified)

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta y_i \mathbf{x}_i$$

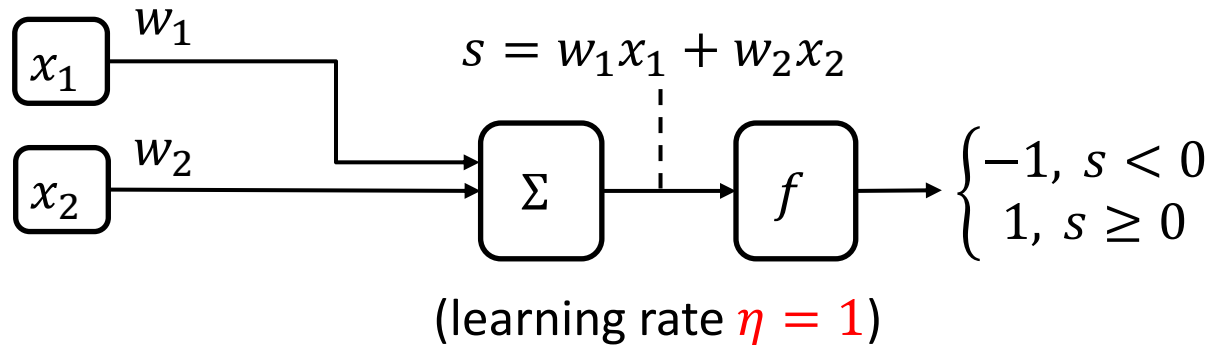
$$k = k + 1$$

Strictly speaking it should be $s_i y_i < 0$ but $\leq$ allows handling the case $\mathbf{w}^{(k)} = \mathbf{0}$

$\mathbf{w}^{(k)}$ represents the value of $\mathbf{w}$ after k updates (useful for theory). If you implement this, just write: $\mathbf{w} = \mathbf{w} + \eta y_i \mathbf{x}_i$

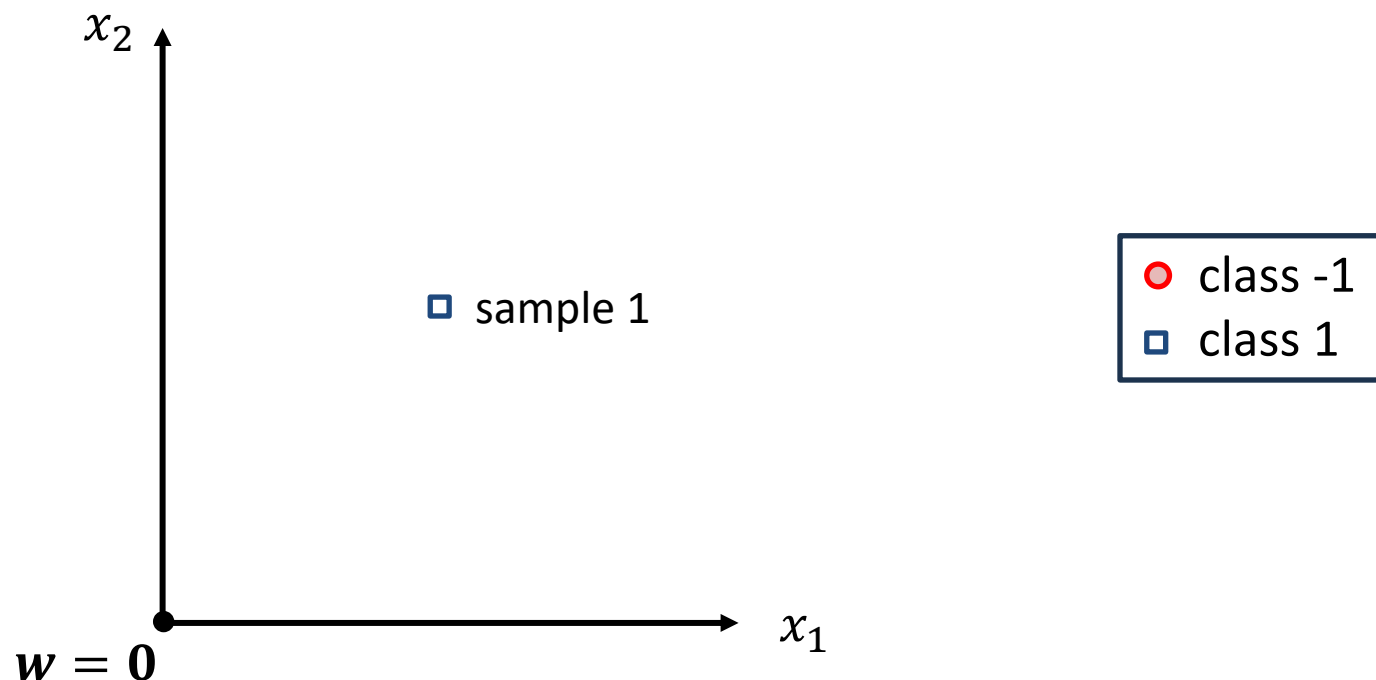
Example 1: Perceptron learning

Basic setup



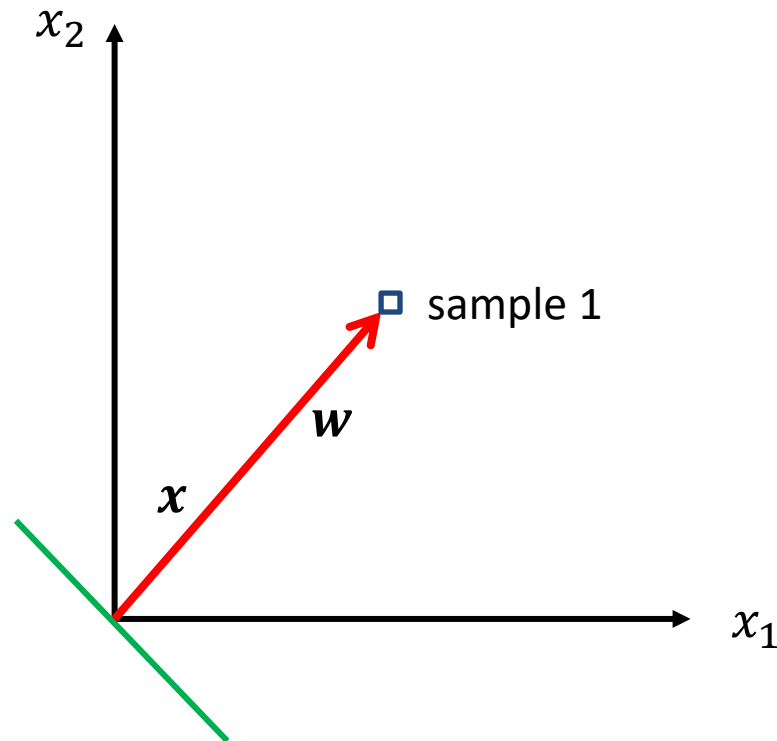
Example 1: Perceptron learning

Start with zero weights, Consider sample 1



Example 1: Perceptron learning

Update

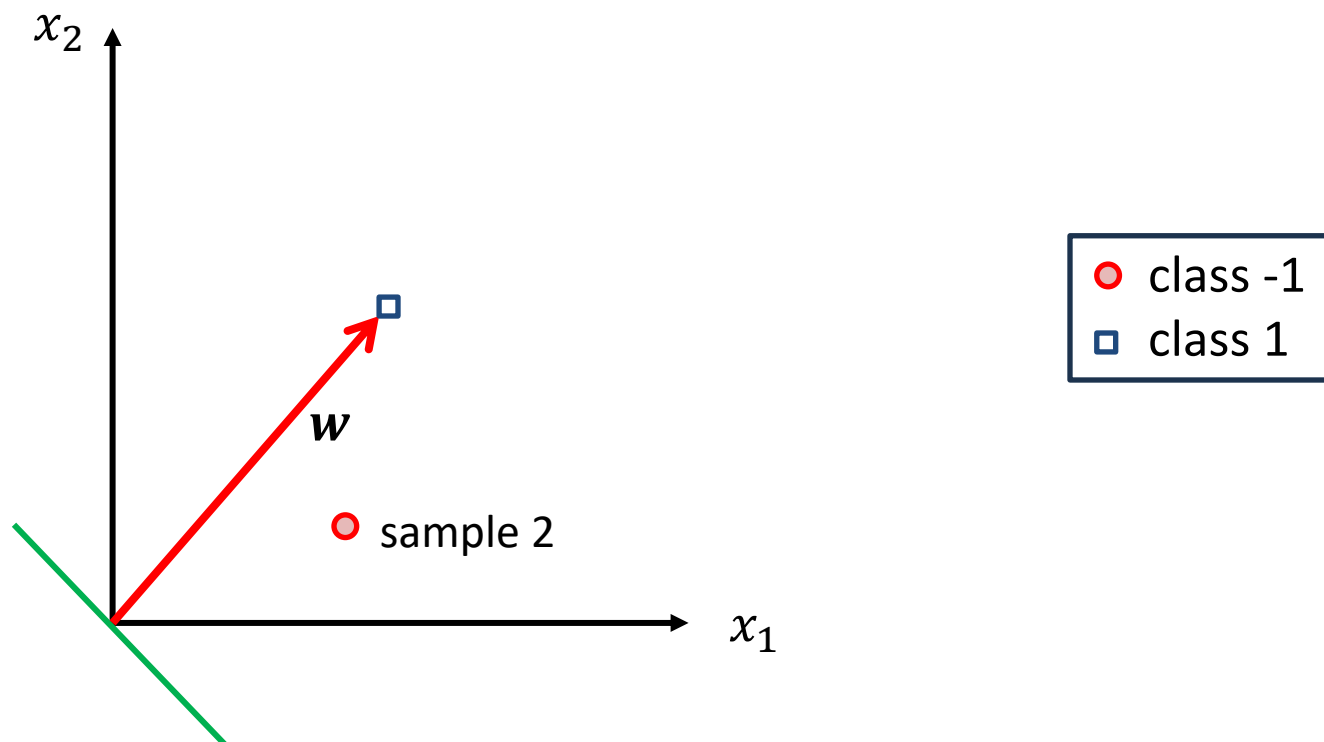


Since $\mathbf{w} = \mathbf{0}$, then update (why?)

$$\mathbf{w} = \mathbf{w} + y\mathbf{x} = \mathbf{x}$$

Example 1: Perceptron learning

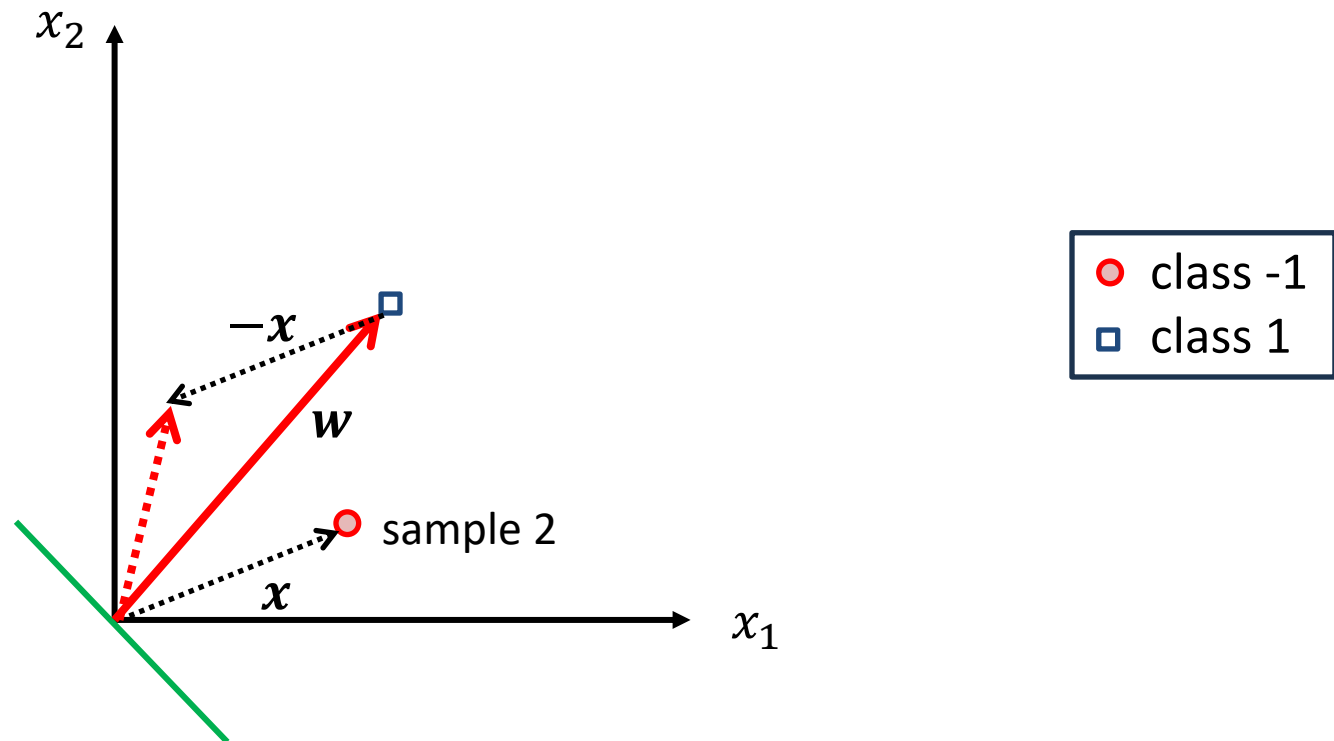
Consider sample 2



Sample 2 is in the wrong side of the decision boundary

Example 1: Perceptron learning

Update

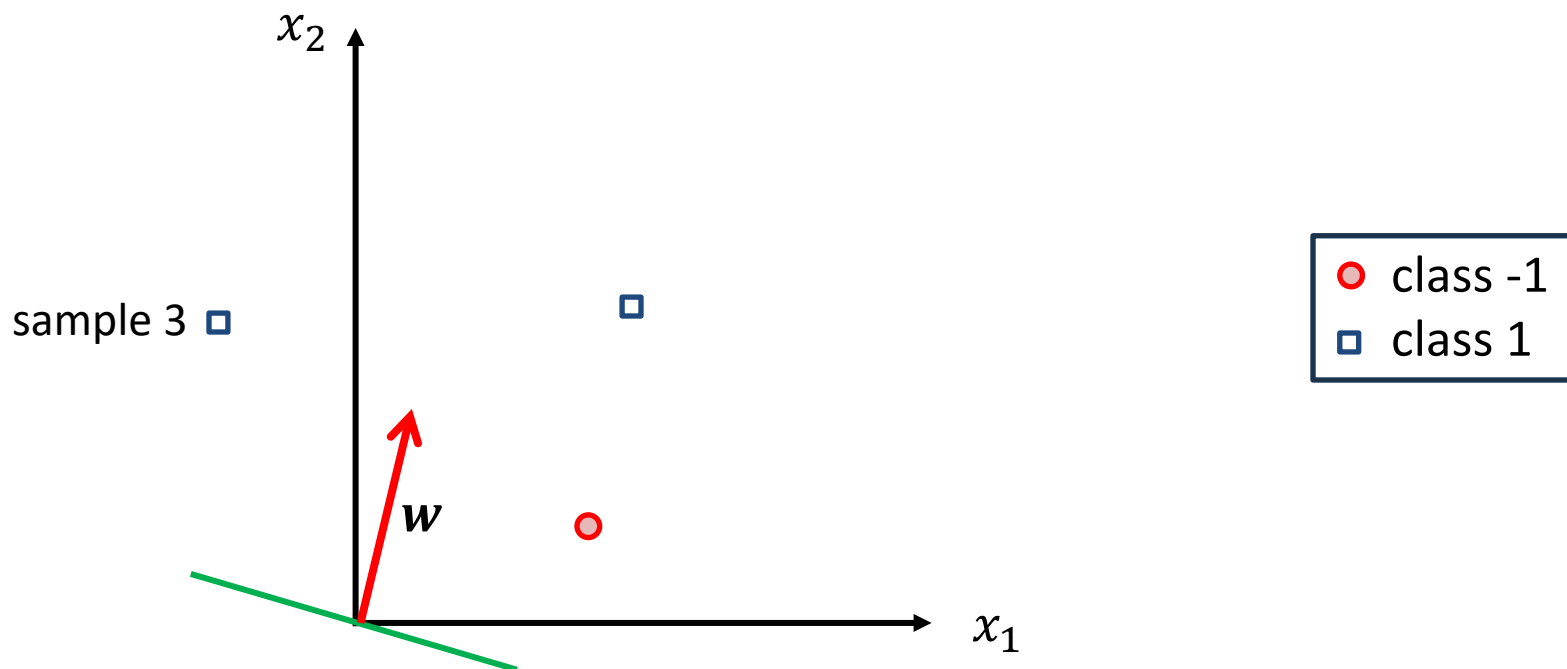


Sample 2 is in the wrong side of the decision boundary, then update

$$w = w + yx = w - x$$

Example 1: Perceptron learning

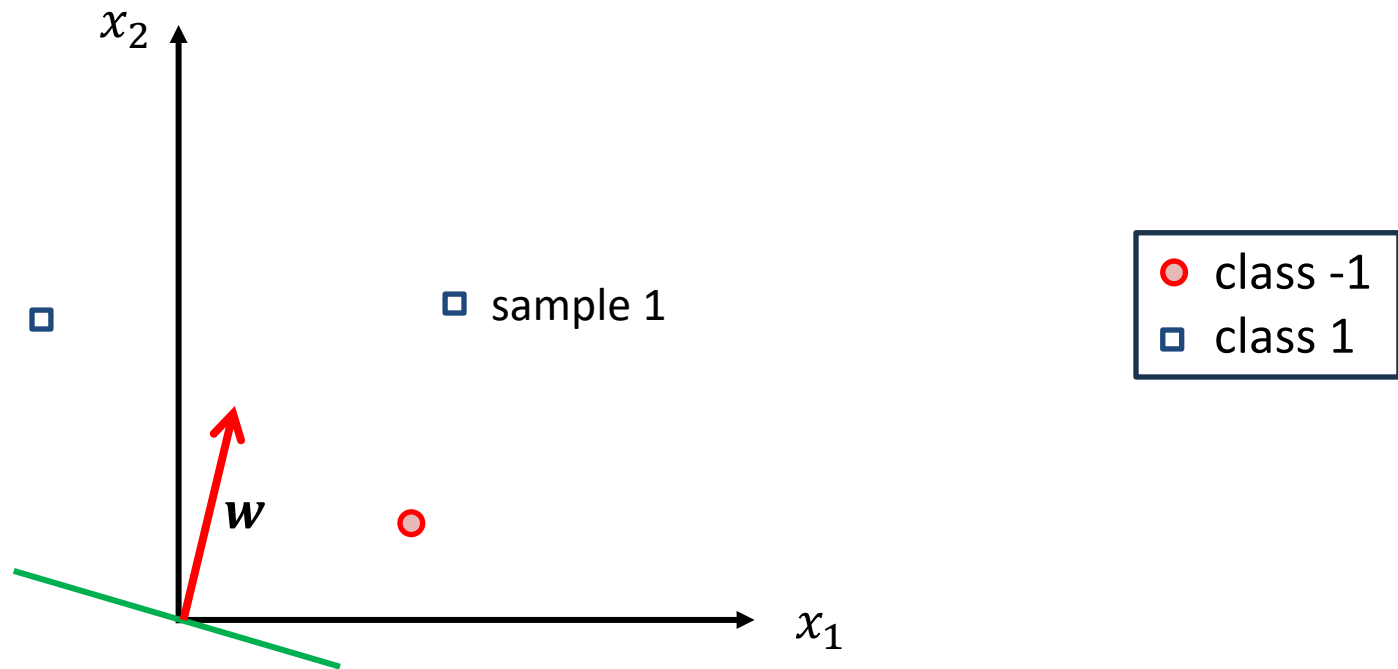
Consider sample 3



Sample 3 is in the correct side of the decision boundary, no update

Example 1: Perceptron learning

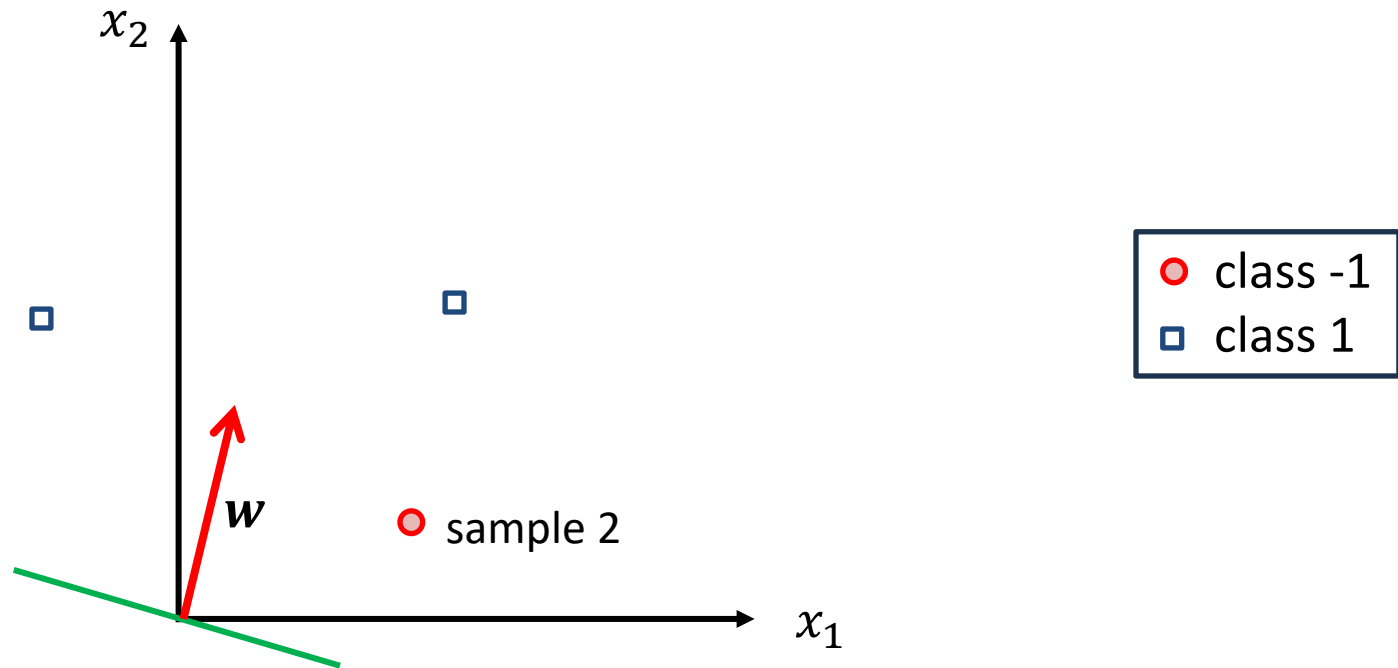
Consider sample 1 again



Sample 1 is in the correct side of the decision boundary, no update

Example 1: Perceptron learning

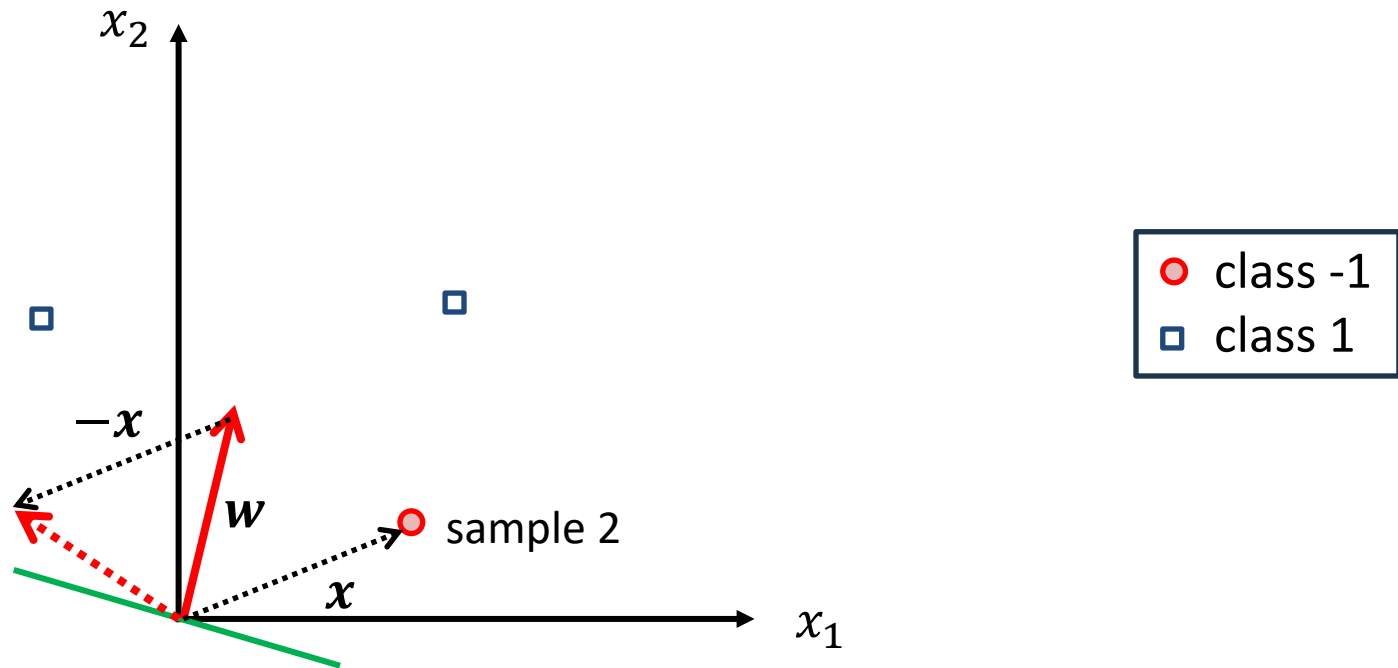
Consider sample 2 again



Sample 2 is in the wrong side of the decision boundary, then update

Example 1: Perceptron learning

Update

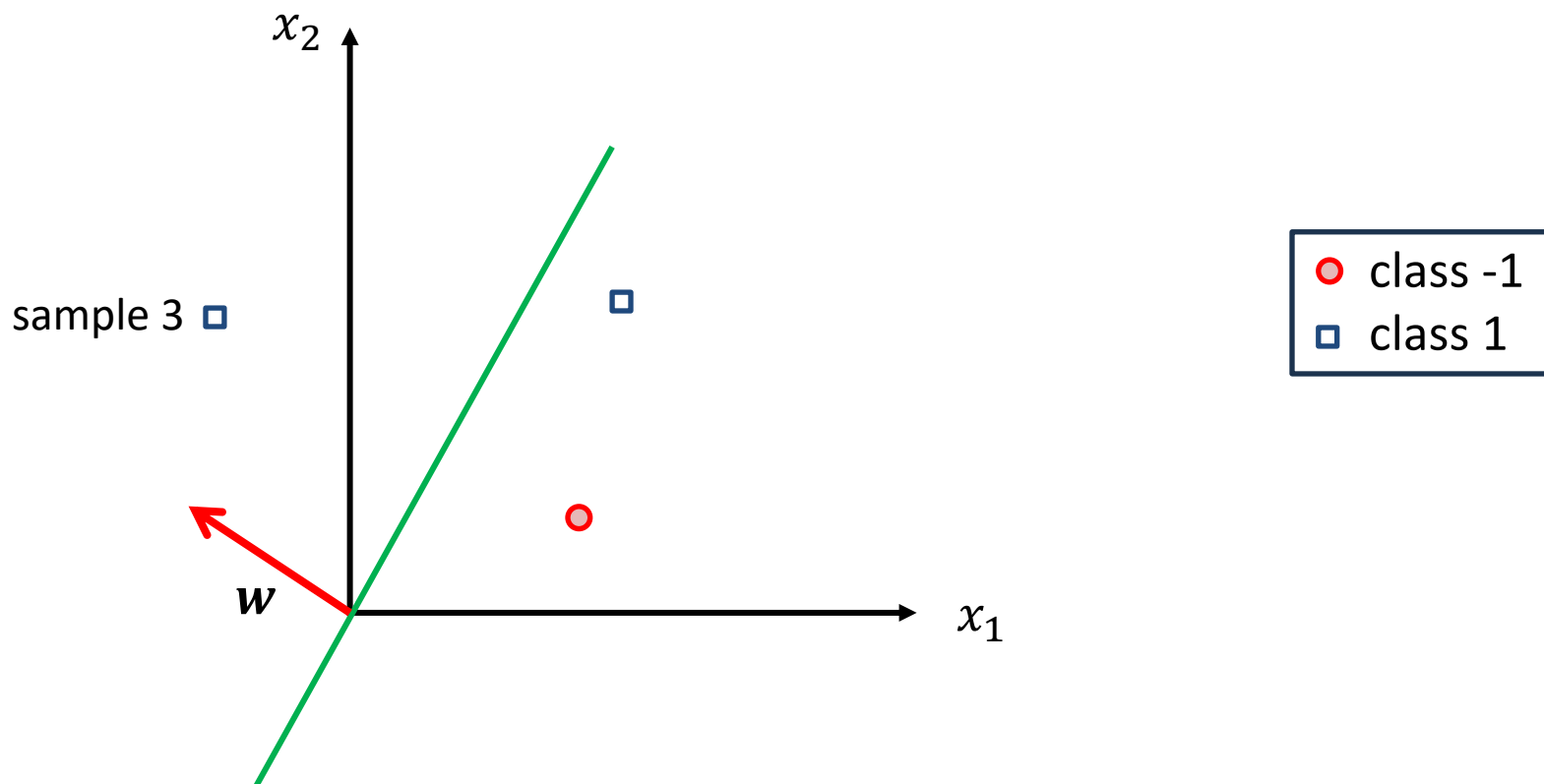


Sample 2 is in the wrong side of the decision boundary, then update

$$\mathbf{w} = \mathbf{w} + y\mathbf{x} = \mathbf{w} - \mathbf{x}$$

Example 1: Perceptron learning

And we continue...



Perceptron training algorithm

Choose initial guess $\mathbf{w}^{(0)}$, $k = 0$

For t from 1 to T (epochs)

For i from 1 to N (training examples)

Compute $s_i = (\mathbf{w}^{(k)})' \mathbf{x}_i$

If $s_i y_i \leq 0$: (sample i misclassified)

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta y_i \mathbf{x}_i$$

$$k = k + 1$$

Convergence Theorem: if the training data is linearly separable, the algorithm is guaranteed to converge to a solution. That is, there exist a finite k such that $L(\mathbf{w}^{(k)}) = 0$

Perceptron convergence theorem

- Assumptions

- * Linear separability: There exists $\mathbf{w}^*$ so that $\frac{y_i(\mathbf{w}^*)' \mathbf{x}_i}{\|\mathbf{w}^*\|} \geq \gamma$ for all training data $i = 1, \dots, N$ and some positive γ .
- * Bounded data: $\|\mathbf{x}_i\| \leq R$ for $i = 1, \dots, N$ and some finite R .

- Proof sketch (for $\eta = 1$)

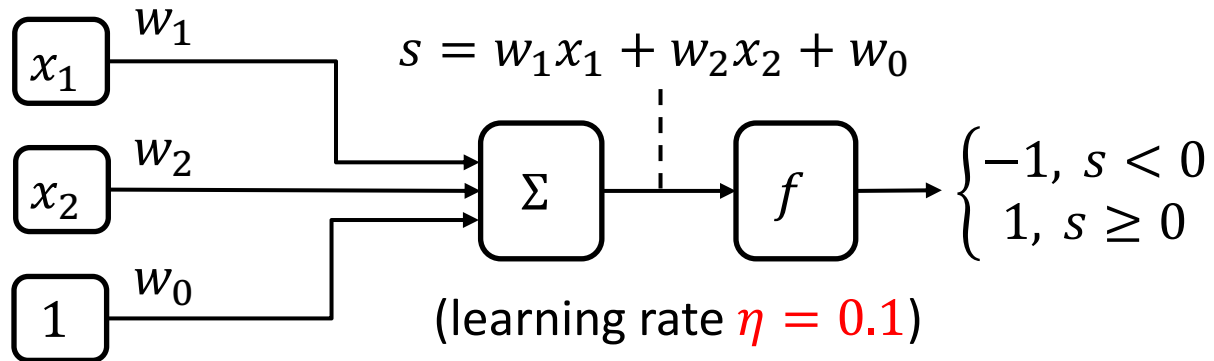
- * Assumes $\mathbf{w}^{(0)} = \mathbf{0}$ to
 - Establish that $(\mathbf{w}^*)' \mathbf{w}^{(k)} \geq k\gamma \|\mathbf{w}^*\|$
 - Establish that $\|\mathbf{w}^{(k)}\|^2 \leq kR^2$
- * Note that $1 \geq \cos(\mathbf{w}^*, \mathbf{w}^{(k)}) = \frac{(\mathbf{w}^*)' \mathbf{w}^{(k)}}{\|\mathbf{w}^*\| \|\mathbf{w}^{(k)}\|} \geq \frac{k\gamma \|\mathbf{w}^*\|}{\|\mathbf{w}^*\| \sqrt{k}R}$
- * Take the left-most and right-most equations $1 \geq \frac{k\gamma \|\mathbf{w}^*\|}{\|\mathbf{w}^*\| \sqrt{k}R}$
- * Rearranging we get $k \leq \frac{R^2}{\gamma^2}$

Pros and cons of perceptron learning

- If the data is linearly separable, the perceptron training algorithm will converge to a correct solution
 - * There is a formal proof $\leftarrow$ good!
 - * It will converge to some solution (separating boundary), one of infinitely many possible $\leftarrow$ bad!
- However, if the data is not linearly separable, the training will fail completely rather than give some approximate solution

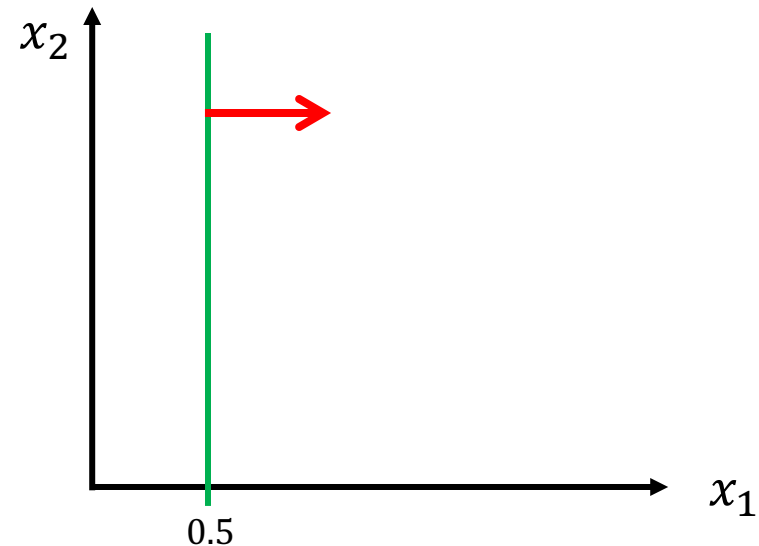
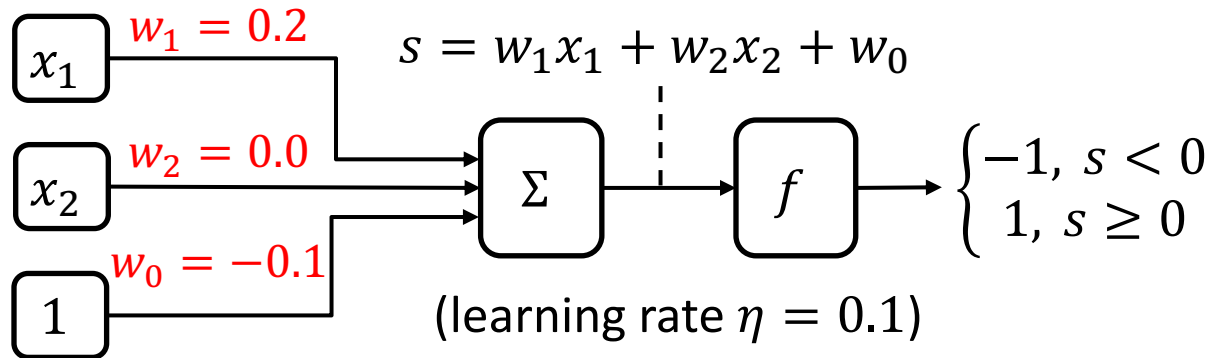
Example 2: Perceptron learning

Basic setup



Example 2: Perceptron learning

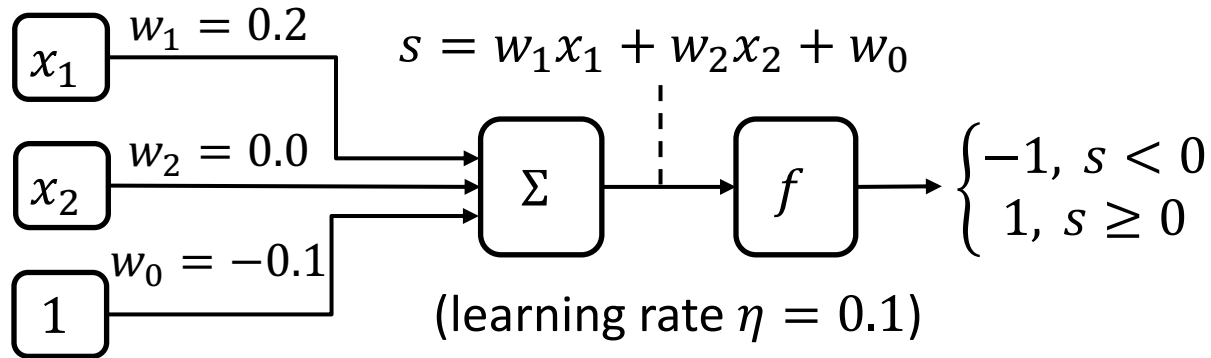
Start with random weights



* We drop the sample index i to have a simpler notation.

Example 2: Perceptron learning

Consider training example 1



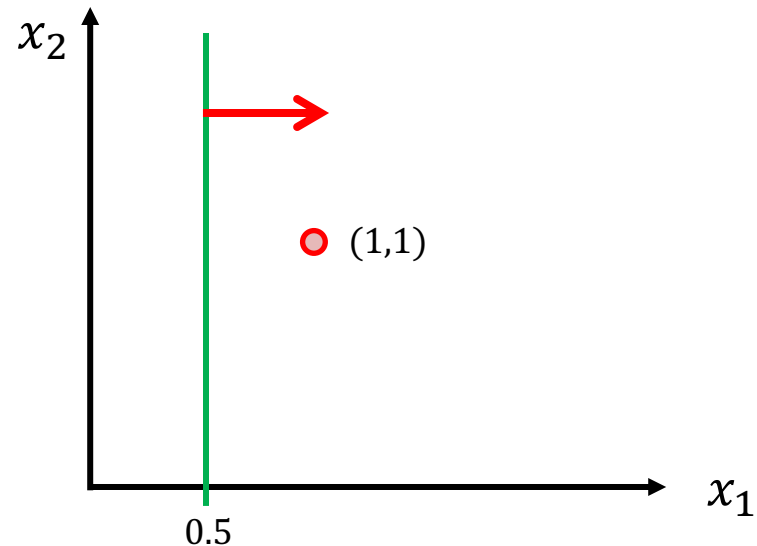
● class -1
■ class 1

$$y(0.2x_1 + 0.0x_2 - 0.1) = -0.1 \leq 0$$

$$w_1 \leftarrow w_1 - \eta x_1 = 0.1$$

$$w_2 \leftarrow w_2 - \eta x_2 = -0.1$$

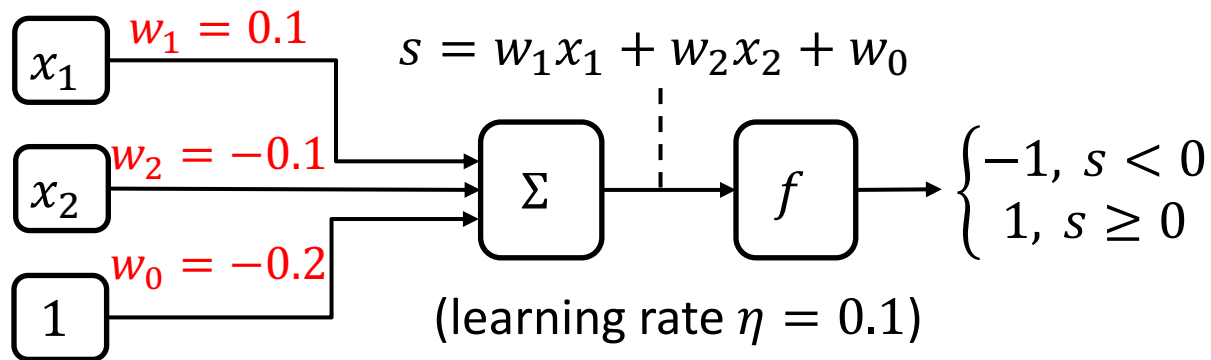
$$w_0 \leftarrow w_0 - \eta = -0.2$$



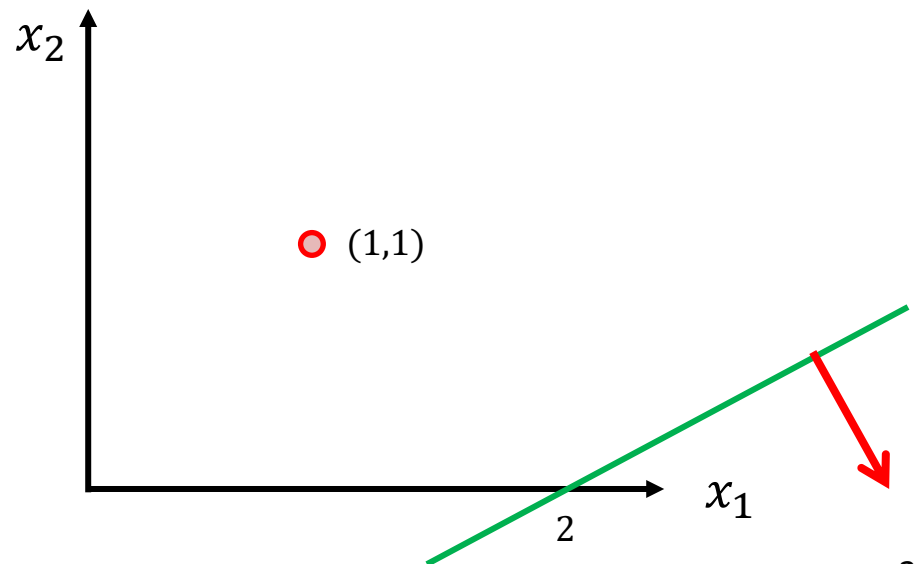
* We drop the sample index i to have a simpler notation.

Example 2: Perceptron learning

Update weights



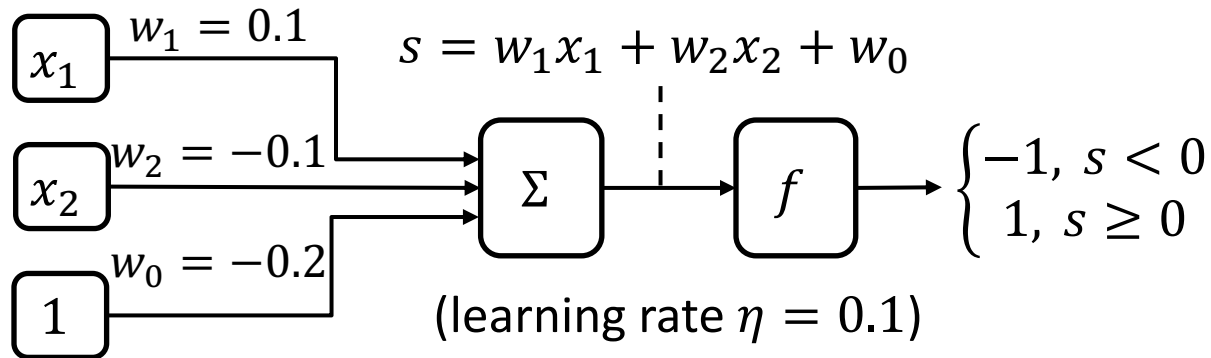
- class -1
- class 1



* We drop the sample index i to have a simpler notation.

Example 2: Perceptron learning

Consider training example 2



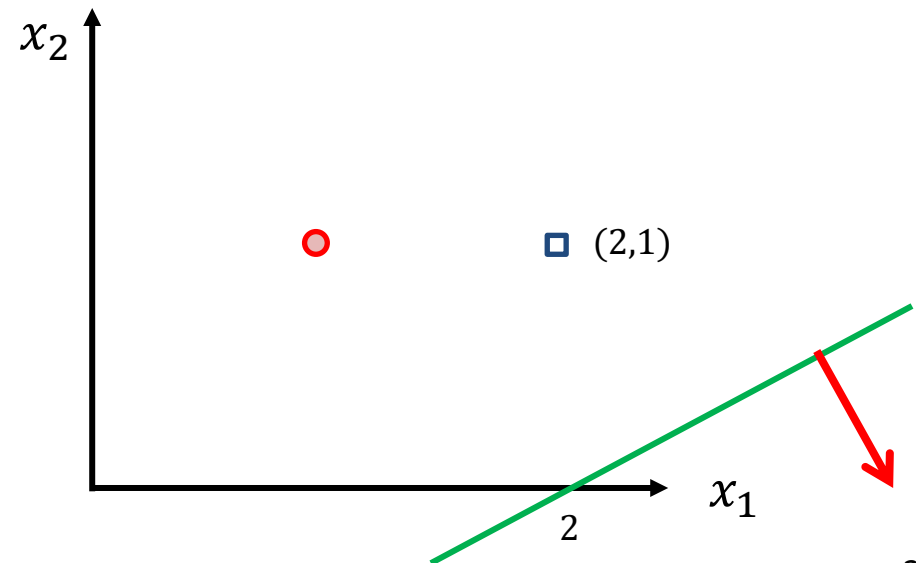
● class -1
■ class 1

$$y(0.1x_1 - 0.1x_2 - 0.2) = -0.1 \leq 0$$

$$w_1 \leftarrow w_1 + \eta x_1 = 0.3$$

$$w_2 \leftarrow w_2 + \eta x_2 = 0.0$$

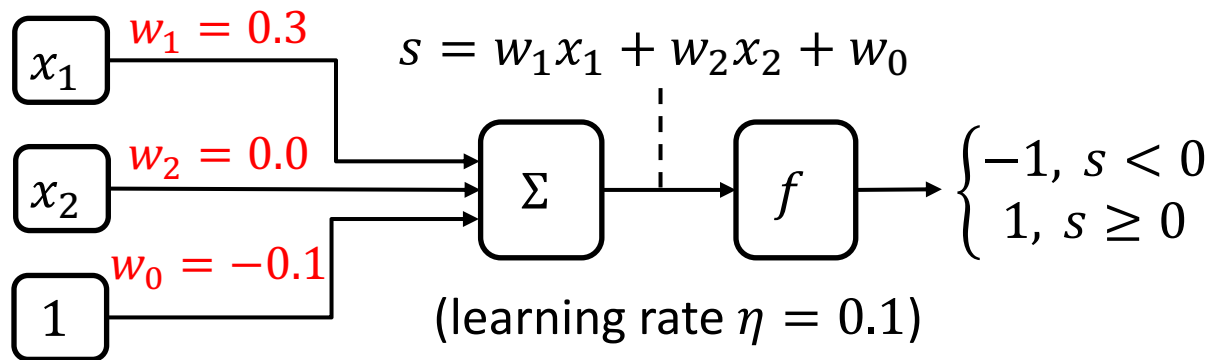
$$w_0 \leftarrow w_0 + \eta = -0.1$$



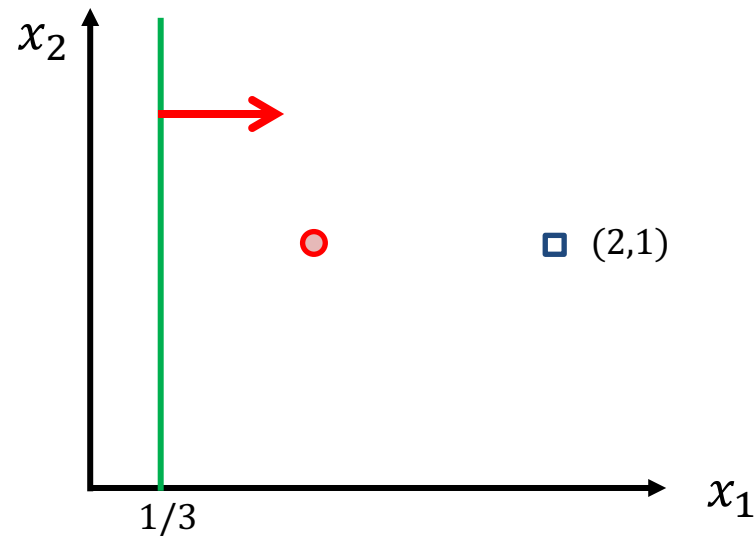
* We drop the sample index i to have a simpler notation.

Example 2: Perceptron learning

Update weights



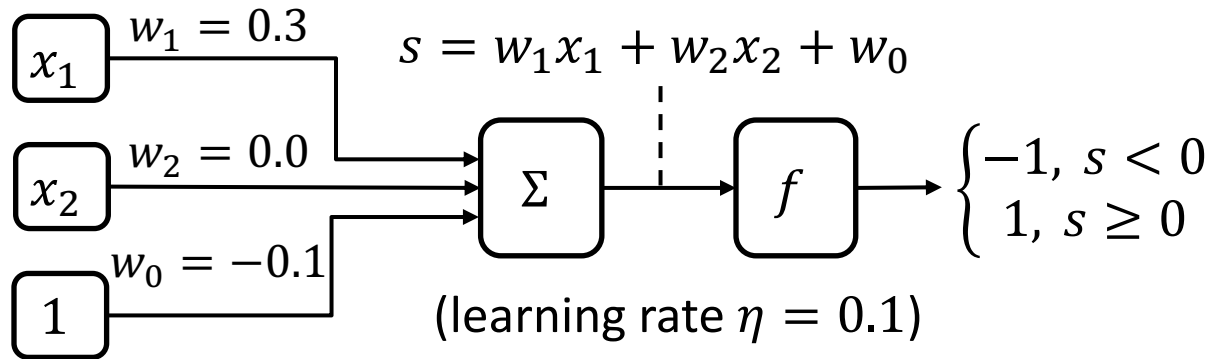
- class -1
- class 1



* We drop the sample index i to have a simpler notation.

Example 2: Perceptron learning

Further examples

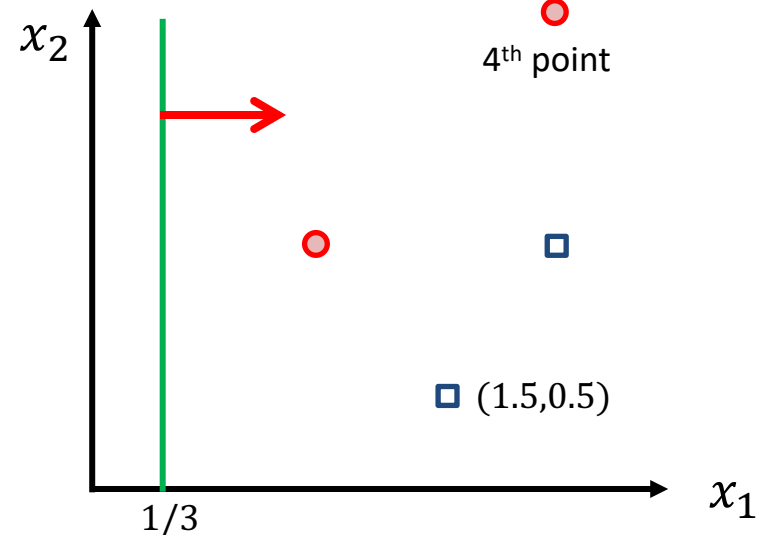


● class -1
■ class 1

$$y(0.3x_1 - 0.0x_2 - 0.1) = 0.35 > 0$$

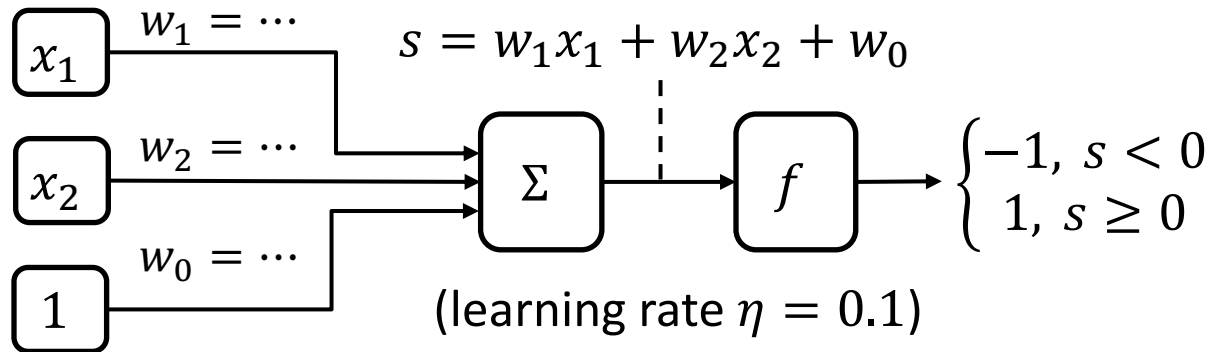
3rd point: correctly classified

4th point: incorrect, update
etc.

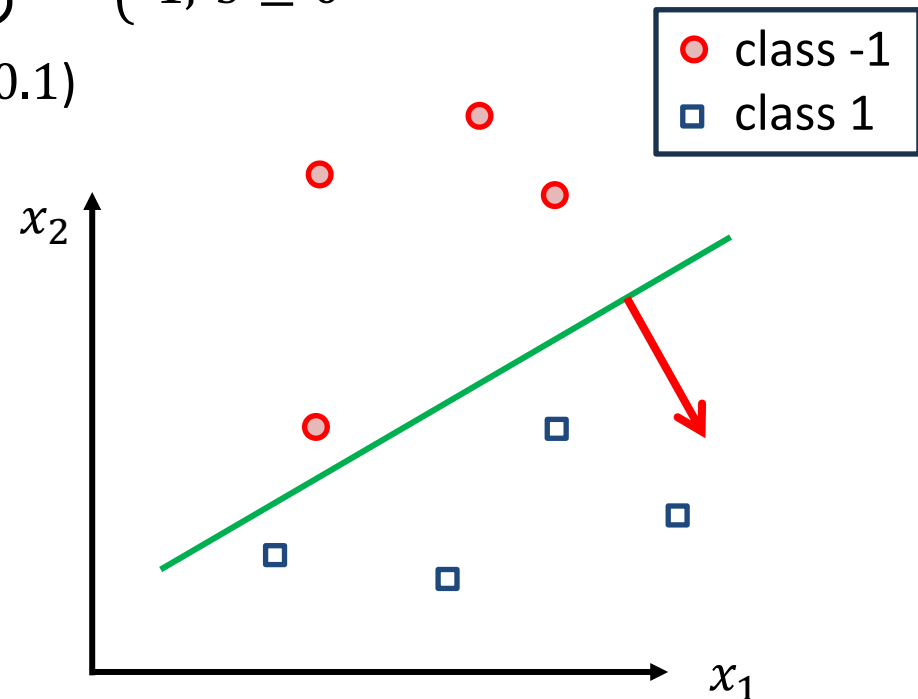


Example 2: Perceptron learning

Further examples



Eventually, all the data will be correctly classified (provided it is linearly separable)



Mini Summary

- Perceptron loss function
- Stochastic gradient descent
- Perceptron training rule
 - * Perceptron convergence theorem

Next: Kernel perceptron

Kernel Perceptron

Another example of a kernelizable learning algorithm (like the SVM).

Perceptron training rule: Recap

Compute $s_i = (\mathbf{w}^{(k)})' \mathbf{x}_i$

If $s_i y_i \leq 0$: (sample i misclassified)

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta y_i \mathbf{x}_i$$

$$k = k + 1$$

Suppose weights are initially set to $\mathbf{w}^{(0)} = \mathbf{0}$

Suppose the algorithm misclassifies sample 1, 7, 29, and 1 again

First update: $\mathbf{w}^{(1)} = \eta y_1 \mathbf{x}_1$

Second update: $\mathbf{w}^{(2)} = \eta y_1 \mathbf{x}_1 + \eta y_7 \mathbf{x}_7$

Third update: $\mathbf{w}^{(3)} = \eta y_1 \mathbf{x}_1 + \eta y_7 \mathbf{x}_7 + \eta y_{29} \mathbf{x}_{29}$

Fourth update: $\mathbf{w}^{(4)} = 2\eta y_1 \mathbf{x}_1 + \eta y_7 \mathbf{x}_7 + \eta y_{29} \mathbf{x}_{29}$ etc.

Accumulating updates: Data enters via dot products

- Weights always take the form $\mathbf{w} = \sum_{j=1}^N \alpha_j \mathbf{y}_j \mathbf{x}_j$, where α some coefficients
- Perceptron weights always **linear comb.** of data!
- Recall that prediction for a new point $\mathbf{x}$ is based on sign of $\mathbf{w}'\mathbf{x}$
- Substituting $\mathbf{w}$ we get $\mathbf{w}'\mathbf{x} = \sum_{j=1}^N \alpha_j \mathbf{y}_j \mathbf{x}_j' \mathbf{x}$
- The dot product $\mathbf{x}_j' \mathbf{x}$ can be **replaced with a kernel**

Kernelised perceptron training rule

Set $\alpha = \mathbf{0}$

For t from 1 to T (epochs)

For i from 1 to N (training examples)

Compute $s_i = \sum_{j=1}^N \alpha_j y_j x'_j x_i$

If $s_i y_i \leq 0$: (sample i misclassified)

$$\alpha_i \leftarrow \alpha_i + \eta$$

($\eta > 0$ is called *learning rate*)

Kernelised perceptron training rule

Set $\alpha = \mathbf{0}$

For t from 1 to T (epochs)

For i from 1 to N (training examples)

Compute $s_i = \sum_{j=1}^N \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i)$

If $s_i y_i \leq 0$: (sample i misclassified)

$$\alpha_i \leftarrow \alpha_i + \eta$$

Mini Summary

- Accumulating weight updates leads to linear combinations of data
- Predictions are dot products with data
- Can replace these with kernel evaluations
- Leads to kernel perceptron with kernel training rule

Next time: Deep learning